

Vue —— 复习要点

1 Vue 简介 (Introduction)

[p.4-6]

1.1 由来 (Origin)

[p.4]

- 作者: 尤雨溪 (Evan You), 2014 年创建 [p.4]
- 从 AngularJS 中提取核心功能 (ViewModel), 打造更轻量、更灵活的框架 [p.4]
- 定位: 用于构建用户界面的**渐进式 JavaScript 框架** (Progressive JavaScript Framework) [p.4]
- 官网: <https://cn.vuejs.org> [p.4]

1.2 核心特性 (Core Features)

[p.5]

- **渐进式 (Progressive)**: 从简单到复杂逐步引入, 降低学习门槛, 适合各种规模项目 [p.5]
- **响应式 (Reactive)**: 数据变化自动更新视图, 避免手动 DOM 操作 [p.5]
- **组件化 (Component-based)**: 将 UI 拆分为独立、可复用的组件, 提高代码复用性 [p.5]
- **模板语法 (Template Syntax)**: 基于 HTML 的模板, 易学易用, 降低学习成本 [p.5]
- **双向绑定 (Two-way Binding)**: v-model 实现数据双向同步, 简化表单处理 [p.5]

1.3 Vue 与 React 对比 (Comparison)

[p.6]

维度	Vue	React
模板语法	HTML-based 模板	JSX
响应式	基于 Proxy 的响应式系统	基于 setState 的状态更新
数据绑定	双向绑定 (v-model)	单向数据流
学习曲线	平缓, 渐进式	中等, 需学习 JSX
生态系统	Vue Router、Pinia、Vue CLI	React Router、Redux、CRA
灵活性	中等	高, 更接近原生 JS

2 Vue 实例 (Vue Instance)

2.1 创建与挂载 (Create & Mount)

- `createApp(App)`: 创建 Vue 应用实例, 接收根组件作为参数
- `app.mount('#app')`: 将应用挂载到 DOM 容器
- `app.use(plugin)`: 安装插件 (Router、Pinia 等)

- `app.component(name, comp)`: 全局注册组件
- `app.config`: 全局配置 (errorHandler、performance 等)
- `app.provide(key, value)`: 应用层 provide, 后代组件可 inject

2.2 选项式 API (Options API) vs 组合式 API (Composition API)

[p.14]

- **选项式 API**: 通过 `data`, `methods`, `computed`, `watch` 等选项组织代码; 适合简单场景或 Vue 2 迁移
- **组合式 API**: 通过 `setup()` 统一组织逻辑, 核心为 `ref()`, `reactive()`, `computed()`, `watch()` [p.14]
- 组合式 API 优势: 更好的逻辑复用 (composables); 更灵活的代码组织; 更好的 TypeScript 类型推断支持
- `<script setup>`: 编译时语法糖, 顶层导入和变量自动暴露给模板, 无需显式 `return`

3 Vue 模板语法 (Template Syntax)

[p.8-12]

3.1 基于 HTML 的模板

[p.8]

- Vue 使用基于 HTML 的模板语法, 相比 JSX 更直观, 更接近传统 HTML, 学习成本更低 [p.8]

3.2 指令系统 (Directive System)

[p.9-12]

指令	作用	示例
v-text	纯文本绑定	
v-html	HTML 内容绑定	<div v-html="htmlContent"></div>
v-bind	属性绑定	
v-on	事件绑定	<button v-on:click="handleClick">
v-model	双向绑定	<input v-model="username" />
v-if	条件渲染	<div v-if="show">显示 </div>
v-for	列表渲染	<li v-for="item in list"> item
v-show	条件显示	<div v-show="visible">可见 </div>

3.3 v-if 与 v-show 对比

[p.11]

场景	v-if	v-show
条件频繁切换	不推荐	推荐
条件很少改变	推荐	不推荐
需要销毁子组件	推荐	不适用

原理:

- v-if: 真正的条件渲染, 切换时销毁/重建 DOM 元素 [p.11]
- v-show: 始终渲染, 仅切换 CSS display 属性 [p.11]

4 Vue 组件与数据传递 (Components & Data Flow)

[p.14-16]

4.1 组件 (Components)

[p.14]

- 组件是 Vue 应用的基本构建块, 将 HTML、CSS、JavaScript 封装在一起, 实现可复用的 UI 单元 [p.14]
- Vue 采用单文件组件 (.vue 文件, Single File Com-

ponents) [p.14]

.vue 文件成员:

成员	作用
<template>	HTML 结构, 只能有一个
<script>	JavaScript 逻辑, setup 是 (Composition API)
<style>	样式, scoped 表示样式只组件

4.2 父组件传子组件: Props

[p.15]

- 父组件通过 props 对象向子组件传递数据 [p.15]

Props 配置选项:

选项	说明
type	指定类型: String, Number, Array, Object, Function
required	是否必填 (Boolean)
default	默认值
validator	自定义验证函数

4.3 子组件传父组件: Emit 事件

[p.16]

- 子组件通过 emit 事件向父组件传递数据 [p.16]
- 子组件触发自定义事件, 父组件监听并响应 [p.16]
- 组合式 API: const emit = defineEmits(['event-name'])
emit('event-name', data)
- 选项式 API: this.\$emit('event-name', payload)

4.4 插槽 (Slots) —— 内容分发

- 默认插槽: <slot></slot>, 父组件传入内容替换 slot 位置
- 具名插槽: <slot name="header"></slot> + <template v-slot:header> (#header 简写)
- 作用域插槽: 子组件通过 slot props 暴露数据给父组件: <slot :item="item"></slot>
- \$slots / useSlots(): 程序化访问插槽内容

5 Vue 响应式系统 (Reactive System)

[p.18-23]

5.1 响应式原理 (Reactivity Principle)

[p.18]

- Vue 的响应式系统是核心特性之一: 当数据变化时, 视图会自动更新 [p.18]
- Vue 3 使用 Proxy 对象拦截数据的读取和写入 [p.18]

工作流程:

- 依赖收集 (Dependency Tracking): 当数据被读取

- 时，建立依赖关系 [p.18]
- **派发更新 (Dispatch Update):** 当数据被修改时，触发视图更新 [p.18]

响应式数据声明方式:

方式	适用类型	钩子函数	触发时机	常用场景
ref()	基本类型 (String, Number, Boolean, ...)	onBeforeMount, onMounted, onBeforeUpdate, onUpdated, onBeforeUnmount, onUnmounted	组件挂载前, 组件挂载后, 组件更新前, 组件更新后, 组件卸载前, 组件卸载后	准备工作, 数据初始化、DOM 操作, 缓存状态, 更新 DOM, 确认操作, 清理资源、取消订阅
reactive()	对象类型 (Object, Array)	onBeforeMount, onMounted, onBeforeUpdate, onUpdated, onBeforeUnmount, onUnmounted	组件挂载前, 组件挂载后, 组件更新前, 组件更新后, 组件卸载前, 组件卸载后	准备工作, 数据初始化、DOM 操作, 缓存状态, 更新 DOM, 确认操作, 清理资源、取消订阅

5.2 计算属性 (Computed Properties)

[p.20-23]

- 计算属性是基于依赖自动计算的值，具有**缓存特性** [p.20]
- 只有当依赖数据变化时，计算属性才会重新计算 [p.23]

computed 与 methods 对比:

对比维度	computed	记忆要点	methods
缓存	依赖变化时重新计算	每次调用都执行	onBeforeMount, onMounted, before/update, before/unmount [p.24]
调用方式	{{ fullName }} 或 computed.fullName	最常用	onMounted, before/unmount [p.24]
返回值	必须返回值	可返回可不返回	onUnmounted 很关键: 清理定时器、取消订阅、释放资源 [p.24]
副作用	不应该有副作用	可以有副作用	onUnmounted 很关键: 清理定时器、取消订阅、释放资源 [p.24]
使用场景	派生值计算、过滤、事件处理、数据格式化		DOM 操作 [p.24]

7 Vue Router (前端路由)

5.3 侦听器 (Watch & WatchEffect)

- **watch(source, callback, options):** 侦听指定数据源，变化时执行回调
- **watchEffect(callback):** 自动追踪回调中的响应式依赖，立即执行一次（无需显式指定依赖）
- **watch** 可获取新旧值: (newVal, oldVal) => { ... }
- **watch 选项:** { deep: true } 深度侦听对象内部变化; { immediate: true } 立即执行回调
- **watch 可侦听:** ref、reactive 对象、getter 函数、或以上类型组成的数组
- **watchEffect** 无法获取旧值，但代码更简洁，自动收集依赖
- **清理副作用:** onCleanup(fn) 注册清理函数（下一次回调执行前调用）
- **停止侦听:** 调用 watch/watchEffect 返回的 stop 函数
- **适用场景:** 异步请求、DOM 操作、手动订阅/取消订阅（computed 无法替代的副作用操作）

6 生命周期钩子函数 (Lifecycle Hooks)

[p.24]

7.1 基本概念

- **Vue 官方路由管理器**，实现单页面应用 (SPA) 的页面导航与视图切换
- **Hash 模式:** URL 带 # (如 /#/about)，兼容性好，无需服务端配置
- **History 模式:** 基于 HTML5 History API (如 /about)，URL 干净，需服务端配置回退
- **创建路由:** createRouter({ history: createWebHistory, routes: [...] })

7.2 核心组件与配置

- **<router-link to="/path">:** 声明式导航链接，默认渲染为 <a> 标签
- **<router-view>:** 路由匹配组件的渲染出口，支持嵌套
- **路由定义:** { path: '/user/:id', component: User, name: 'user', meta: {...} }
- **动态路由:** /user/:id, 通过 \$route.params.id 或 useRoute().params.id 获取参数
- **查询参数:** \$route.query 或 useRoute().query (响应式对象)
- **命名路由:** router.push({ name: 'user', params: { id: 1 } })

7.3 程式化导航 (Programmatic Navigation)

- `router.push(path)`: 导航到新页面, 有历史记录 (可返回)
- `router.replace(path)`: 替换当前页面, 无历史记录 (不可返回)
- `router.go(n)`: 前进/后退 `n` 步; `router.back()` / `router.forward()`

7.4 导航守卫 (Navigation Guards)

- **全局守卫**: `router.beforeEach((to, from, next) => {...})` (前置)、`beforeResolve` (解析)、`afterEach` (后置)
- **路由独享守卫**: `beforeEnter` (在路由配置中定义)
- **组件内守卫**: `beforeRouteEnter`、`beforeRouteUpdate`、`beforeRouteLeave`
- 解析流程: 导航触发 → 组件内守卫 → 路由独享守卫 → 全局守卫 → 组件渲染

7.5 进阶特性

- **路由懒加载**: `component: () => import('./User.vue')`, 按需加载, 优化首屏
- **嵌套路由**: `children` 配置, 父组件中放置 `<router-view>`
- **命名视图**: 同一路由同时展示多个视图 (如 `sidebar + main`)
- **路由元信息 (meta)**: 传递附加数据, 常用于权限控制、面包屑、页面标题

8 状态管理: Vuex 与 Pinia (State Management)

8.1 Vuex (Vue 2 官方方案)

- 核心概念: State、Getters、Mutations、Actions、Modules
- **State**: 全局状态, 单一状态树 (Single State Tree) —— `store.state.xxx`
- **Getters**: 派生状态 (类似 `computed`) —— `store.getters.xxx`
- **Mutations**: 同步修改 `state` 的唯一方式 —— `store.commit('mutation', payload)`
- **Actions**: 处理异步操作 —— `store.dispatch('action', payload)`; 通过 `commit` 间接修改 `state`
- **Modules**: 模块化拆分 `store`, 支持命名空间 (`namespaced: true`)

- 辅助函数: `mapState`, `mapGetters`, `mapMutations`, `mapActions`
- 严格模式 (`strict: true`): 开发环境下非 `mutation` 修改 `state` 会报错

8.2 Pinia (Vue 3 推荐方案)

- Vue 3 官方推荐状态管理库, 替代 Vuex
- 核心: `defineStore(id, options)` 定义 `store`
- **State**: 函数返回初始状态 (支持 `ref/reactive`)
- **Getters**: 类似 `computed` 的函数, 接收 `state` 参数
- **Actions**: 函数, 可直接修改 `state` (同步/异步均可), 相当于 Vuex 的 `mutations + actions` 合并
- **无 Mutations**: 简化架构, `actions` 直接操作 `state`
- **扁平化**: 无 `modules` 嵌套, 用多个独立 `store` 替代
- 两种风格: Options Store (`{ state, getters, actions }`) 与 Setup Store (组合式 API 风格)
- 完整 TypeScript 类型推断; DevTools 调试 + HMR 热更新
- `$reset()`: 重置 `state` 到初始值; `$patch()`: 批量更新 `state`

8.3 Vuex 与 Pinia 对比

- Pinia 无 `mutations`, API 更简洁; Vuex 强制 `mutation` 同步约束
- Pinia 完整 TypeScript 支持; Vuex 类型推断较弱
- Pinia 无嵌套 `modules`, 用扁平 `store`; Vuex 需 `modules` 拆分
- Pinia 支持组合式 API 风格 (Setup Store); Vuex 仅选项式
- 推荐: 新项目用 Pinia; Vue 2 项目可迁移至 Pinia (兼容)

9 前端其他知识 (Other Frontend Knowledge)

[p.26-42]

9.1 TypeScript

[p.27-31]

- **JavaScript 的痛点**: 动态类型语言, 缺乏类型检查, 大型项目中难以维护 [p.27]
- **TypeScript**: Microsoft 于 2012 年发布, 是 JavaScript 的超集 (Superset), 添加了可选的静态类型系统 (Static Type System) [p.28]

- TypeScript 可以与 React 和 Vue 一起使用 [p.30–31]
- 优势：编译时类型检查、更好的 IDE 支持、提升代码可维护性 [p.28]

9.2 服务端渲染 (Server-Side Rendering, SSR)

[p.33–36]

- **为什么需要 SSR：**解决纯客户端渲染 (CSR) 的首屏加载慢、SEO 不友好等问题 [p.33]
- Web 软件系统架构的发展演变：从传统服务端渲染 (MPA)，到客户端渲染 (SPA/CSR)，再到现在的主流方案——服务端渲染 (SSR) 或静态生成 (SSG) [p.34–36]
- SSR 方案：Next.js (React)、Nuxt.js (Vue) [p.34–36]

9.3 性能优化 (Performance Optimization)

[p.38–42]

(1) 代码层面优化

[p.38–40]

- 合理使用 `v-if` / `v-show`：频繁切换用 `v-show`，条件稳定用 `v-if` [p.38]
- 使用 `computed` 代替 `methods` 处理派生数据（利用缓存）[p.38]
- 使用 `v-for` 时添加唯一 `key`，提升列表渲染效率 [p.38]
- 组件懒加载/异步组件 (Async Components)：按需加载，减少首屏包体积 [p.39]
- 避免不必要的响应式数据：非响应式数据不要用 `ref/reactive` 包裹 [p.39]
- 合理拆分组件，避免单个组件过于庞大 [p.40]

(2) 资源优化

[p.41]

- 图片懒加载、压缩、使用 WebP 格式 [p.41]
- CSS/JS 代码压缩与合并 [p.41]
- 使用 CDN 加载静态资源 [p.41]
- Tree-shaking：移除未使用的代码 [p.41]

(3) 网络层面优化

[p.42]

- 减少 HTTP 请求数 [p.42]

- 使用浏览器缓存 (HTTP 缓存策略) [p.42]
- 启用 Gzip/Brotli 压缩 [p.42]
- 使用 HTTP/2 多路复用 [p.42]

10 本章小结 (Chapter Summary)

[p.43]

Vue 核心知识框架：

1. **Vue 简介：**渐进式框架，由 Evan You 创建，核心特性包括响应式、组件化、模板语法、双向绑定 [p.4–5]
2. **Vue 实例：**`createApp` 创建 + `mount` 挂载；选项式 API vs 组合式 API (Composition API)；`<script setup>` 语法糖 [p.14]
3. **模板语法：**8 大指令 (`v-text`, `v-html`, `v-bind`, `v-on`, `v-model`, `v-if`, `v-for`, `v-show`)；`v-if` 与 `v-show` 使用场景区分 [p.9–11]
4. **组件与数据传递：**单文件组件 (`.vue`)；Props 父传子 (`type/required/default/validator`)；Emit 子传父；Slots 插槽 [p.14–16]
5. **响应式系统：**Vue 3 基于 Proxy 拦截；`ref` (基本类型) vs `reactive` (对象)；`computed` 缓存计算 vs `methods`；`watch/watchEffect` 侦听器 [p.18–23]
6. **生命周期钩子：**三对钩子函数 (`before/mounted`, `before/update`, `before/unmount`)；`onMounted` 和 `onUnmounted` 最为常用 [p.24]
7. **Vue Router：**Hash/History 模式；`<router-link>` + `<router-view>`；动态路由、嵌套路由；编程式导航 (`push/replace/go`)；导航守卫 (`beforeEach` 等)；路由懒加载
8. **Vuex / Pinia：**Vuex 五核心 (State/Getters/Mutations)，Pinia 三核心 (State/Getters/Actions)，无 mutations，扁平 store，推荐 Vue 3 项目使用
9. **前端其他知识：**TypeScript (JS 超集 + 静态类型系统)；SSR (服务端渲染解决 CSR 首屏/SEO 问题)；性能优化 (代码 + 资源 + 网络三个层面) [p.27–42]